

Deep Q-Learning for High-Frequency Market Making

Antoine Mazet & Léo Pommier
Reinforcement Learning Project — April 10, 2026

1 Introduction

A market maker simultaneously quotes a *bid* and an *ask* price, earning the **spread** in exchange for providing continuous liquidity to other participants. This activity carries two fundamental risks: **inventory risk** (accumulating a directional position in a depreciating asset) and **adverse selection** (being executed against by better-informed traders who anticipate price movements).

We train a **DQN (Deep Q-Network)** agent on a synthetic Limit Order Book (LOB) to learn an optimal quoting policy. Its performance is evaluated against two benchmarks: the analytical **Avellaneda–Stoikov** model, which represents a closed-form risk-averse solution, and a **random quoting policy**, which serves as a performance floor.

The project is organised into the following modules:

Module	Role
<code>env/data_lob.py</code>	Stochastic LOB simulator
<code>env/MM_env.py</code>	Gymnasium-compatible trading environment
<code>dqn_agent.py</code>	DQN: network, replay buffer, exploration
<code>train_dqn.py</code>	Training loop
<code>eval.py</code>	Unified evaluation framework

2 LOB Simulation

2.1 Train / Evaluation Split

Two independent datasets are generated with different random seeds to prevent look-ahead bias: seed 42 (100 000 steps) for training and seed 999 (50 000 steps) for out-of-sample evaluation. The agent never observes the evaluation price path during learning, which is critical to verify its generalisation capability.

2.2 Mathematical Models

The simulator combines six stochastic mechanisms to produce realistic synthetic market data.

(1) Spread — Ornstein–Uhlenbeck mean-reversion:

$$s_t = s_{t-1} + \kappa(\bar{s} - s_{t-1}) + \sigma_s \varepsilon_t \quad (1)$$

where $\bar{s}$ is the long-run mean spread, κ the reversion speed, and σ_s the spread volatility.

(2) Mid-price with order-book imbalance:

$$m_t = m_{t-1} + \sigma_m \eta_t + k_{\text{imb}} \frac{q_t^b - q_t^a}{q_t^b + q_t^a} \quad (2)$$

A heavy bid side ($q^b \gg q^a$) exerts upward pressure on the price, capturing a well-documented microstructure effect.

(3) Order flow — Poisson processes:

$$N \sim \text{Poisson}(\lambda \Delta t) \quad (3)$$

Three event types occur at each step: limit orders L_t (add liquidity), market orders M_t (consume liquidity), and cancellations C_t (remove resting orders).

(4) Queue dynamics:

$$q_t = q_{t-1} + L_t - M_t - C_t \quad (4)$$

(5) **State-dependent intensities:** arrival rates adapt to market conditions — large spread $\Rightarrow \lambda_L \downarrow, \lambda_M \uparrow$; sparse book $\Rightarrow \lambda_L \propto (q_0 - q) \uparrow$; dense book $\Rightarrow \lambda_C \propto q/q_0 \uparrow$.

(6) Price formation:

$$\text{bid}_t = m_t - \frac{s_t}{2}, \quad \text{ask}_t = m_t + \frac{s_t}{2} \quad (5)$$

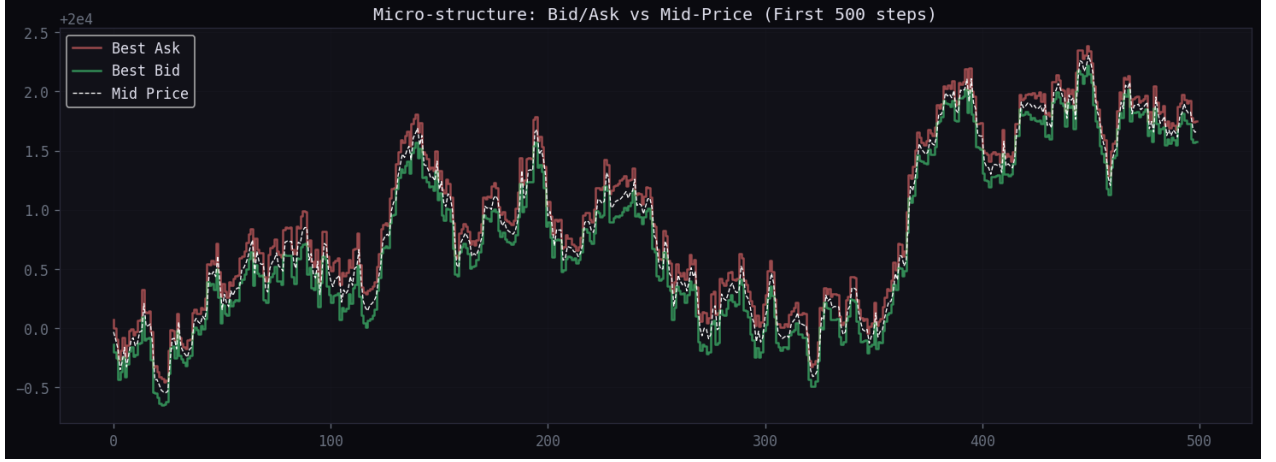


Fig. 1: Micro-structure dynamics of the synthetic Limit Order Book (LOB) over the first 500 simulation steps. The plot illustrates the evolution of the *Best Ask* (red) and *Best Bid* (green) prices, with the *Mid-Price* shown as a dashed white line. This high-frequency view highlights the discrete price movements and the persistent bid-ask spread, which serves as the fundamental environment for the market-making agent.

2.3 Exploratory Data Analysis (Out-of-Sample, Seed 999)

Three key statistical properties are examined before any strategy evaluation:

- **Mid-price trajectory:** GBM-like dynamics with mean-reverting episodes, covering uptrend, downtrend, and sideways regimes.
- **Spread distribution:** concentrated near $\bar{s}$, with tails representing high-volatility periods requiring wider quoting.
- **Returns distribution:** slight fat tails (excess kurtosis) reflecting price-jump dynamics — the primary source of adverse-selection risk.

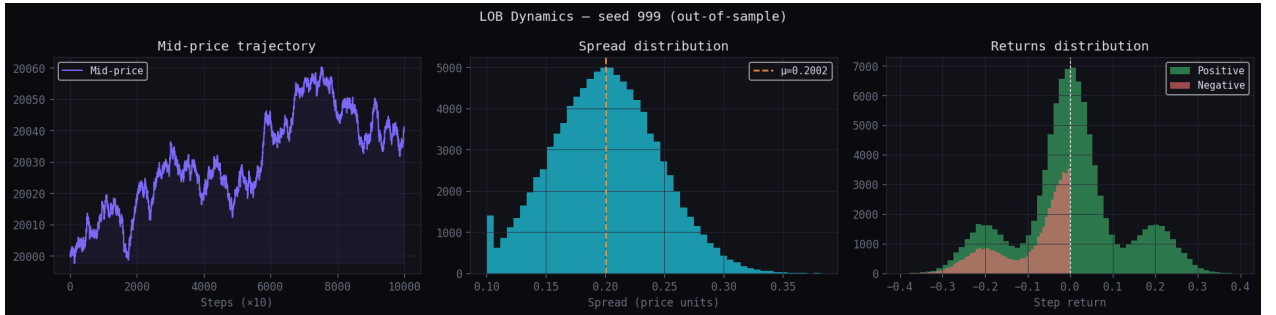


Fig. 2: Statistical characterisation of the out-of-sample LOB (seed 999). Fat tails in returns confirm the presence of adverse-selection risk.

The annotated liquidity map (Figure 3) provides a microscopic view of the order book: large Level-1 volumes mark support/resistance zones; their sudden disappearance signals a trade execution; liquidity migrating from deeper levels to Level 1 indicates increasing urgency and often precedes a directional price move.



Fig. 3: Annotated liquidity map. Bubble size encodes order volume at each price level. Deep-level liquidity walls are predictive signals for future state transitions.

3 DQN Agent

3.1 MDP Formulation

Market making is modelled as a finite-horizon MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$.

State: $s_t = (m_t, s_t, q_t^b, q_t^a, \text{inv}_t)$ — mid-price, spread, best-level queue sizes, and current inventory.

Actions: discrete quoting offsets (δ^b, δ^a) from the mid-price, the size of the grid actions is $5 * 5 = 25$:

$$\text{bid}_t^{\text{agent}} = m_t - \delta^b, \quad \text{ask}_t^{\text{agent}} = m_t + \delta^a.$$

Reward: gross profit penalised for directional exposure:

$$r_t = \Delta \text{PnL}_t - \alpha |\text{inv}_t| - \beta \text{inv}_t^2 \quad (6)$$

Objective: $\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t \right].$

3.2 Architecture and Key Components

Q-network. The online network maps a state s to a vector of Q-values one per discrete action through two fully-connected layers of width 256 with ReLU activations. $Q(s, a)$ estimates the expected cumulative discounted reward of committing to action a in state s and subsequently following the greedy policy.

A naive DQN uses the *same* network both to *select* the greedy action and to *evaluate* its value when computing Bellman targets:

$$y_t^{\text{DQN}} = r_t + \gamma Q_{\text{target}} \left(s_{t+1}, \arg \max_a Q_{\text{target}}(s_{t+1}, a) \right) \quad (7)$$

This coupling introduces a systematic upward bias: the maximisation operator preferentially selects actions whose Q-values are overestimated by random approximation errors, and the same errors then inflate the target itself. Over thousands of updates, these biases compound and destabilise learning.

Double DQN breaks this feedback loop by decoupling the two roles across the online and target networks:

$$y_t = r_t + \gamma Q_{\text{target}}\left(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta)\right) \quad (8)$$

The *online* network θ selects the action; the *target* network θ^- evaluates it. Because the two parameter sets diverge after each gradient step, overestimation in one cannot directly amplify the other, yielding more accurate value estimates, stabler training, and ultimately a better-performing policy.

ε -greedy exploration. Exponential decay:

$$\varepsilon_t = \varepsilon_{\min} + (\varepsilon_{\max} - \varepsilon_{\min}) e^{-t/\tau_\varepsilon} \quad (9)$$

ensuring broad exploration early in training and exploitation of the learned policy later.

Experience replay. Transitions (s_t, a_t, r_t, s_{t+1}) are stored in a fixed-capacity buffer; random mini-batch sampling breaks temporal correlations and improves sample efficiency.

Double DQN with target network. A periodically synchronised target network Q_{target} provides stable Bellman targets. Action selection (online network) is decoupled from action evaluation (target network) to reduce Q-value overestimation:

$$y_t = r_t + \gamma Q_{\text{target}}\left(s_{t+1}, \arg \max_a Q(s_{t+1}, a)\right) \quad (10)$$

Algorithm 1 Double DQN — Learning Step

- 1: Sample mini-batch $\mathcal{B}$ from the replay buffer
 - 2: **for** each $(s_i, a_i, r_i, s'_i) \in \mathcal{B}$ **do**
 - 3: $a^* \leftarrow \arg \max_a Q(s'_i, a)$ *(online network selects)*
 - 4: $y_i \leftarrow r_i + \gamma Q_{\text{target}}(s'_i, a^*)$ *(target network evaluates)*
 - 5: **end for**
 - 6: Minimise $\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_i \text{HuberLoss}(Q(s_i, a_i), y_i)$; clip gradients
 - 7: Every N_{update} steps: $Q_{\text{target}} \leftarrow Q$
-

4 Baselines

Avellaneda–Stoikov (A&S). A closed-form analytical model that computes an indifference price adjusted for current inventory and remaining episode time. It represents a sophisticated, risk-averse quoting strategy and serves as our primary non-RL benchmark.

Random policy. Bid and ask offsets are drawn uniformly at random from the action space. It establishes the performance floor and quantifies the cost of operating without any strategic structure.

5 Experiment 1 — Avellaneda–Stoikov vs. Random

Both strategies are evaluated on the out-of-sample LOB (seed 999) over 2000-step episodes. Three metrics are tracked: cumulative risk-adjusted reward, inventory level, and raw PnL.



Fig. 4: Single-episode comparison of Avellaneda–Stoikov and Random strategies (2000 steps, seed 999).

Strategy	Cum. Reward	Raw PnL	Penalty	Mean inv
Avellaneda–Stoikov	−2 557.9	+302.2	+2 860.1	1.0
Random	−2 379 455.3	+134.5	+2 379 589.8	43.3

Table 1: Baseline strategy evaluation on the out-of-sample LOB (seed 999, 2000-step episode). Cumulative reward = Raw PnL − Penalty.

Cumulative reward. A&S decays far more slowly than Random. The steep downward slope of the random baseline quantifies how quickly unmanaged inventory costs overwhelm a trading account.

Inventory dynamics. A&S displays mean-reverting behaviour, continuously skewing its quotes to attract inventory-reducing fills. The Random policy follows a near-random walk, regularly drifting to ± 60 units and accumulating significant directional exposure.

Raw PnL. Both strategies generate positive gross profit at various points, confirming that the LOB provides consistent spread-capture opportunities. A&S captures this profit more consistently and with lower variance.

Key insight for DQN training. The strong correlation between inventory spikes and reward drops is the central signal. To outperform A&S, the DQN must learn a *neutralisation reflex*: tightening the spread on one side to encourage inventory-reducing fills.

6 Experiment 2 — DQN vs. Avellaneda–Stoikov vs. Random

6.1 Training Configuration

Hyperparameter Value	
Hidden layers	[256, 256], ReLU
Replay buffer	10 000 transitions
Batch size	64
Discount γ	0.99
Target update	Every 500 steps
$\varepsilon_{\max}$ / $\varepsilon_{\min}$ / τ_{ε}	1.0 / 0.01 / 1000
Optimiser / Loss	Adam / Huber

6.2 Results



Fig. 5: Three-way strategy comparison over 2000 out-of-sample steps (seed 999).

Strategy	Cum. Reward	Raw PnL	Penalty	Mean inv
DQN (Trained)	-11 484.8	+414.0	+11 898.8	2.5
Avellaneda–Stoikov	-37 975.8	+228.0	+38 203.9	4.2
Random	-749 864.0	+276.0	+750 140.0	21.6

Table 2: Three-way strategy evaluation on the out-of-sample LOB (seed 999, 2 000-step episode), sorted by cumulative reward. Cumulative reward = Raw PnL – Penalty.

Raw PnL — the DQN alpha. The DQN ends above 400, while A&S stagnates around 230. The agent has discovered microstructure signals — likely related to **imbalance** or **local volatility** — that the rigid A&S formula cannot exploit. It also finds a better equilibrium between spread width and fill rate.

Inventory — total control. While Random drifts to -60 units and A&S oscillates between -20 and +10, the DQN keeps its position in a very tight corridor around zero. Whenever a position is taken, the agent adjusts its quotes almost instantly to unwind it — an *inventory fear* that is the hallmark of top high-frequency market makers.

Cumulative reward — the RL advantage. The DQN is the only strategy whose reward curve remains near zero or slightly positive: its gross profit is not consumed by inventory penalties. The gap versus Random is dramatic; DQN also edges out A&S, whose wider inventory swings trigger greater penalty accumulation.

7 Conclusion

This study demonstrates that a **Double DQN** agent can outperform both the Avellaneda–Stoikov analytical framework and a random policy in a simulated high-frequency market-making environment. Three conclusions emerge:

1. **DQN superiority.** The agent achieves higher raw PnL *and* exemplary inventory stability, converting gross profit into net profit without incurring penalty costs.
2. **Risk management is decisive.** The Random baseline collapses due to uncontrolled directional exposure, proving that inventory management is a prerequisite for profitability.
3. **Adaptability over rigidity.** The DQN learns finer microstructure correlations (imbalance, local volatility) that the fixed A&S formula cannot capture, allowing more aggressive spread capture with strict inventory discipline.

Perspectives

The results presented here open several promising research directions.

Richer state representations. Incorporating deeper LOB snapshots, multi-level trade flow imbalance, or short-horizon volatility estimates could allow the agent to detect adverse-selection signals earlier and adjust its quotes before incurring losses.

Continuous action spaces. Replacing the DQN with a **Soft Actor-Critic** or **TD3** architecture would allow fine-grained, continuous quote placement — particularly valuable in tight-spread regimes where a single tick separates a profitable fill from a losing one.

Realistic execution modelling. Introducing pro-rata queue priority, partial fills, and latency would bring the environment closer to live exchange mechanics and stress-test the robustness of the learned policy under conditions it has never encountered.

Live deployment. Bridging simulation and production requires a calibration layer that continuously fits the LOB parameters $(\kappa, \bar{s}, \sigma_m, \lambda)$ to real tick data, paired with a risk guardrail module that overrides the RL policy when inventory or drawdown thresholds are breached.

These extensions chart a path from a proof-of-concept agent to a deployable, adaptive system — one that combines the rigour of analytical finance with the flexibility of model-free reinforcement learning.